

多级供应链下的库存优化问题

——随机需求下的策略研究

第八组 张子乐 吴贵义

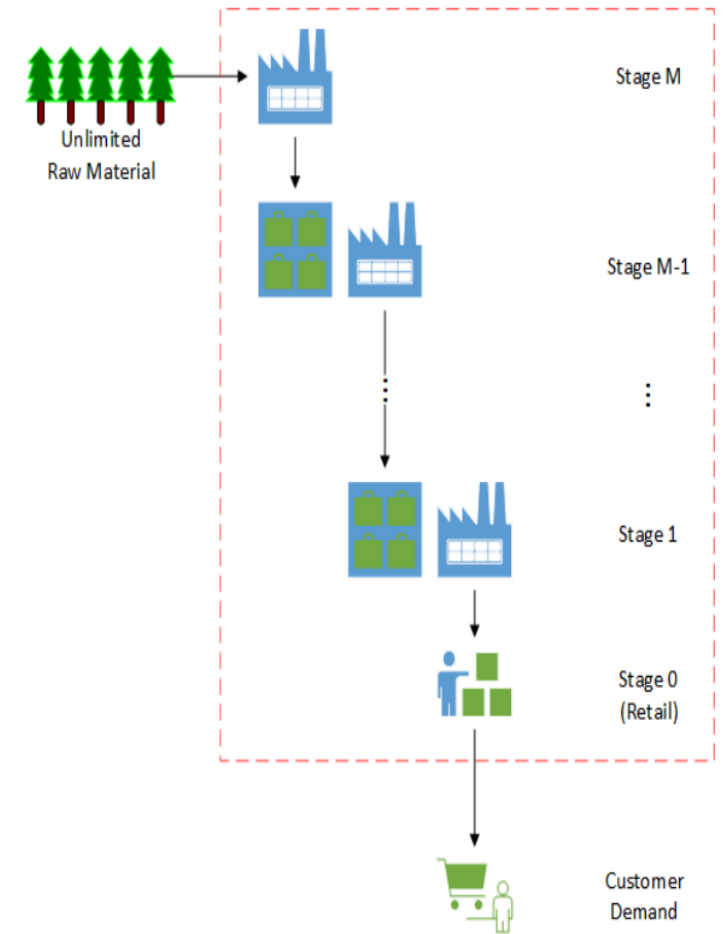
2026.4.23

1. 多级供应链下库存优化的背景与问题定义
2. 数据生成
3. 算法设计 (Base Stock + SHLP +PPO)
4. 算法求解结果展示
5. 性能分析
6. 系统优化
7. 总结

库存管理是现代供应链的核心环节之一：

- The more inventory you have, the better you can satisfy your customer's requests
- High inventory levels come at a cost(仓储成本)

★ Trade-off between service level and holding costs



- 📌 随机扰动：客户需求量不确定
- 📌 Backlog：当天无法满足需求 ➡ 延期满足&利润减少
- 📌 Lead time：上游节点制造与运输产品到下游节点的时间
- ⚠️ Bullwhip effects：下游微小的需求波动导致上游产生巨大的库存波动



定义一个包含 4 个节点的供应链系统（零售商、批发商、分销商和工厂）。零售商不能生产，工厂没有库存，原材料无限。每个节点都需要在每个周期决定向其上游节点订购多少货物，以满足其下游节点的需求。

每个周期依次发生 6 个步骤：

- 下游节点向上游下达补给订单
- 接受补给，库存增加(lead time)
- 零售商根据库存满足客户需求
- 处理缺货情况(backlog)
- 记录剩余库存并计算仓储成本
- 周期末库存清零



- (决策变量)订货量 $\hat{R}_t^m$: 节点 m 在周期 t 向上游节点发送的期望补货量。
- (状态变量)现有库存量 I_t^m : 节点 m 在周期 t 期初实际持有的可用库存。
- (状态变量)在途库存 T_t^m : 节点 m 已经成功下达, 但因存在提前期 L_m , 仍在途尚未到达的货物量。
- (状态变量)缺货量 U_t^m : 节点 m 未能满足其下游发来的补货请求量 ($U_t^m = \hat{R}_t^{m-1} - S_t^m$)。对于零售商 ($m = 0$), 可能转化为累积缺货量 B_t^0 。

- 产能与库存约束：节点 m 实际获批的发货量 R_t^m 受到上游产能 c^{m+1} 、上游当前库存 I_t^{m+1} 的限制：

$$R_t^m = \min \left(c^{m+1}, I_t^{m+1}, \hat{R}_t^m \right)$$

- 库存守恒： $t + 1$ 时刻的期初库存 = t 时刻的期初库存 + 经过提前期后到达的货物量 - 本期的实际发货量：

$$I_{t+1}^m = I_t^m + R_{t-L_m}^m - S_t^m$$

- 在途库存守恒：

$$T_{t+1}^m = T_t^m - R_{t-L_m}^m + R_t^m$$

- 实际发货量限制:

- 对于上游节点 ($m > 0$) , 实际发货量即为批准给下游的量: $S_t^m = R_t^{m-1}$
- 对于零售商 ($m = 0$) , 发货量受限于当前总可用库存与客户总需求的极小值:

$$S_t^0 = \min(I_t^0 + R_{t-L_m}^0, D_t + B_{t-1}^0)$$

在有限的运行周期 T 内，最大化整个供应链系统的折现总利润 $\mathbb{E}[\sum P_t^m]$
节点 m 单个周期 t 的利润计算由以下几部分组成（折现因子 α ）：

- 销售收益：单价 $\times$ 实际销售发货量 ($p^m S_t^m$)
- 采购成本：采购单价 $\times$ 实际获批的订货量 ($r^m R_t^m$)
- 缺货惩罚：单位缺货罚金 $\times$ 缺货量 ($k^m U_t^m$)
- 仓储成本：单位仓储成本 $\times$ 期末库存量 ($h^m I_{t+1}^m$)

 目标函数可表示为最大化期望累积收益：

$$\max \mathbb{E} \left[\sum_{t=1}^T \sum_{m=0}^M \alpha^t (p^m S_t^m - r^m R_t^m - k^m U_t^m - h^m I_{t+1}^m) \right]$$

⚙ 假设顾客的需求量 D_t 服从参数为 $\lambda = 20$ 的泊松分布：

$$P(D_t = k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad (k = 0, 1, 2, \dots)$$

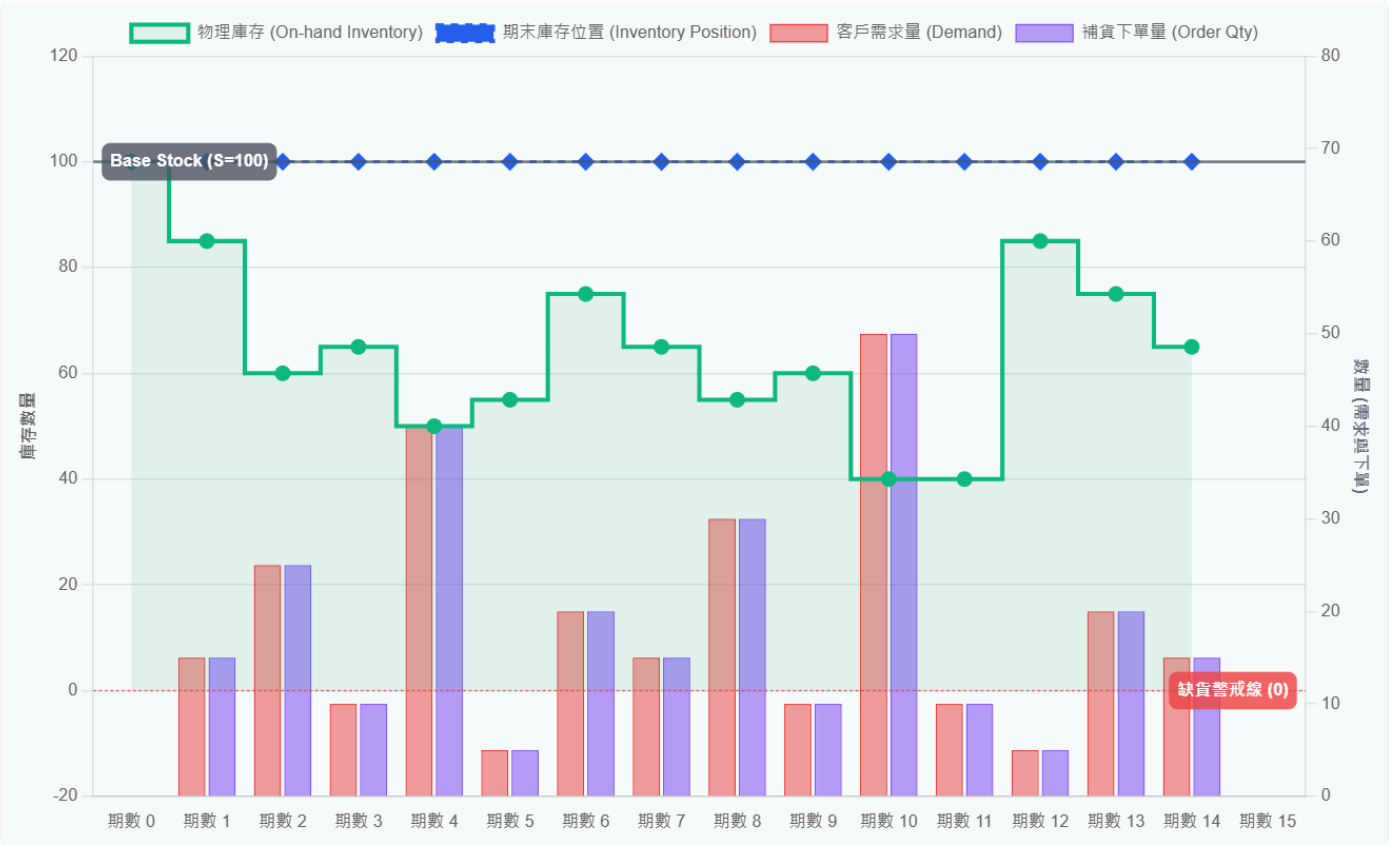
Parameter	Symbol	Stage 0	Stage 1	Stage 2	Stage 3
Initial Inventory	I_0	100	100	200	-
Unit Sales Price	p	\$2.00	\$1.50	\$1.00	\$0.75
Unit Replenishment Cost	r	\$1.50	\$1.00	\$0.75	\$0.50
Unit Backlog Cost	k	\$0.10	\$0.075	\$0.05	\$0.025
Unit Holding Cost	h	\$0.15	\$0.10	\$0.05	-
Production Capacity	c	-	100	90	80
Lead Times	L	3	5	10	-

The base stock policy has been shown to be **optimal** for capacitated production-inventory systems under **certain conditions** and it's one of the valid OR approaches used in practice due to its **simplicity**.

✨ The requested reorder quantity is given by:

$$\hat{R}_t^m = \max \left(0, z^m - \sum_{m'=0}^m \left(I_t^{m'} + T_t^{m'} - B_{t-1}^{m'} \right) \right)$$

where z^m is the **base-stock level** at stage m and the term in the summation is the current **inventory position** at the beginning of period t .



期數 (T)	需求 (D)	到貨量 (ARRIVAL)	物理庫存 (ON-HAND)	在途庫存 (PIPELINE)	庫存位置 (IP)	補貨下單 (ORDER QTY)
期數 1	15	+0	85	15	100	下單 15
期數 2	25	+0	60	40	100	下單 25
期數 3	10	+15	65	35	100	下單 10
期數 4	40	+25	50	50	100	下單 40
期數 5	5	+10	55	45	100	下單 5

- Infinitesimal Perturbation Analysis (IPA) ✗

基于梯度下降, 目标函数不光滑/不可微 $\rightarrow$ 线搜索(Wolfe Condition)寻找最优步长的方法常常不能收敛

- Derivative-free Optimization (DFO) ✓

(Based on Powell's methods)不依赖梯度, 以初始值 z 为起点, 构建共轭方向进行启发式搜索, 最后 *rounding/local search*

- Simulated Annealing (SA) ✓

如果 $\Delta E < 0$ 则接受, 如果 $\Delta E > 0$ 则以概率 $\exp\{-\Delta E/T\}$ 接受

基于 Powell's method 的 DFO 算法对初始值较为敏感:

$$z^m \approx \mu \cdot T_m + 2\sigma\sqrt{T_m}, \quad T_m = \sum_{i=0}^m L_i + 1$$

忽视产能限制/缺货惩罚, 表现不佳 ✖

泊松分布的真实参数:

$$\mu = \sigma^2 = 20$$

✔ 参数估计:

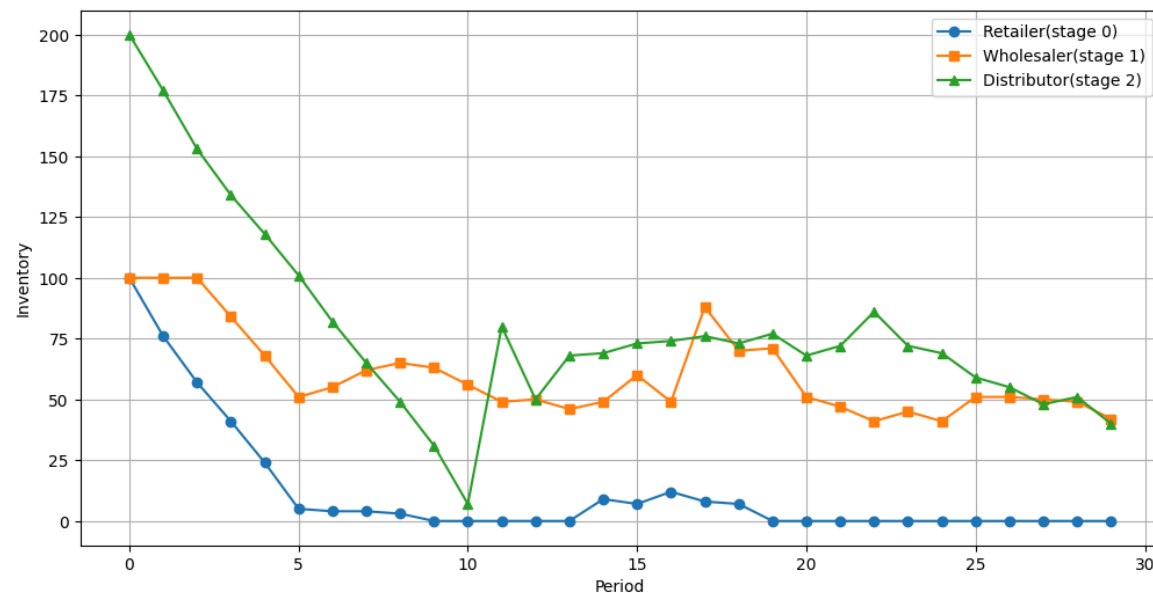
$$\hat{\mu} = 20.14, \hat{\sigma} = 4.66$$

```
from scipy.optimize import minimize
# Powell's method
result_DFO = minimize(
    fun=evaluate_z, x0=z0,
    args=(env, 20), method='Powell',
    options={'xtol': 1.0, 'ftol': 1.0, 'maxiter': 30, 'disp': True})
```

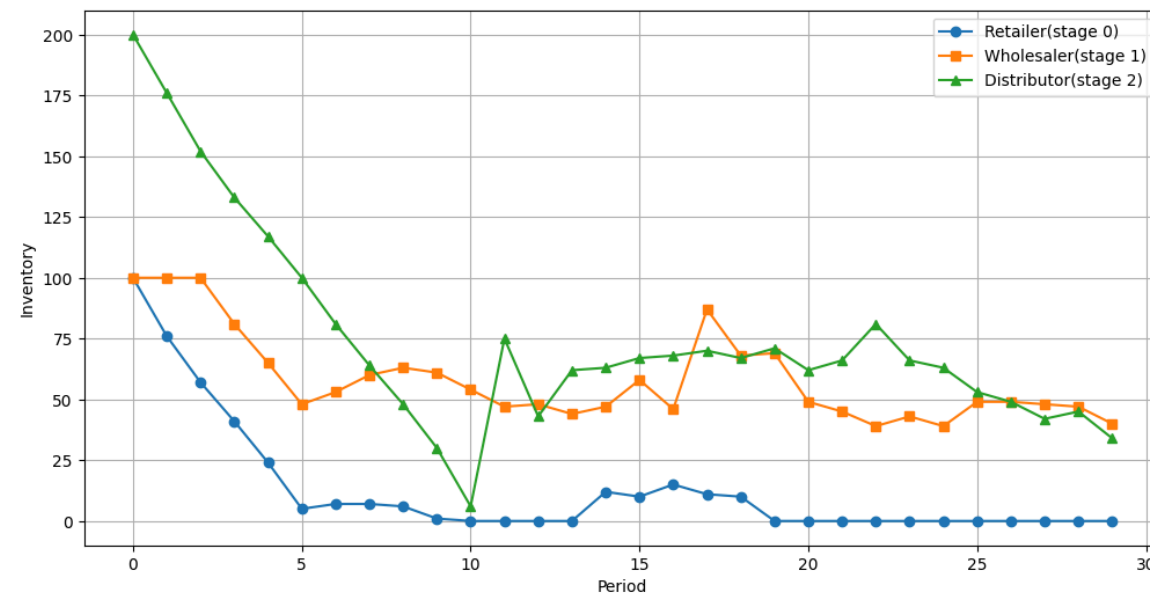
模拟退火算法产生新解：

```
numpy.random.randint(-5,6,size=env.num_stages-1) # SA算法产生新解
```

参数	参数值	含义	参数	参数值	含义
T0	5000	初始温度	t_min	0.1	终止温度
rate	0.99	降温速率	iter	20	内循环迭代次数



左图: *DFO(rounding)* 单次模拟库存变化曲线

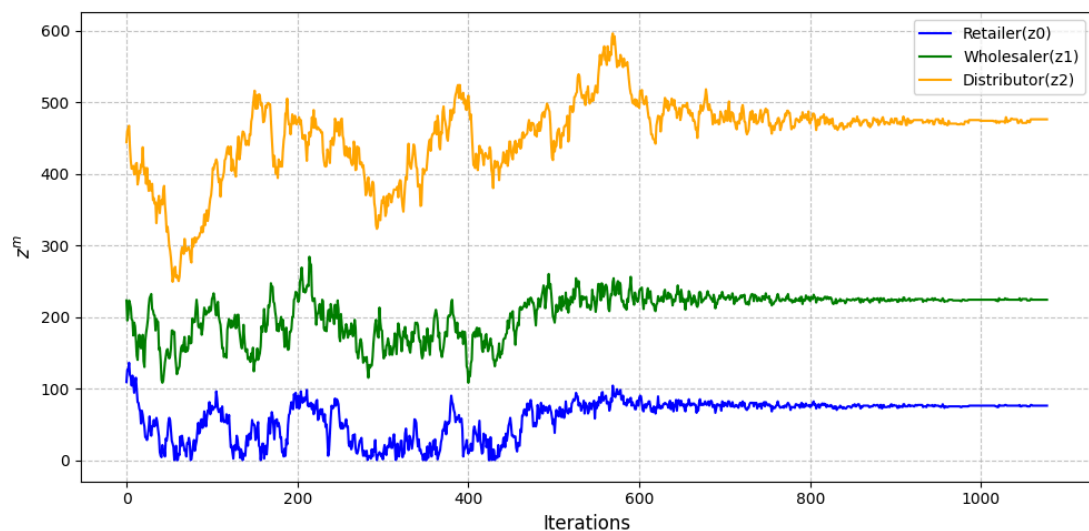
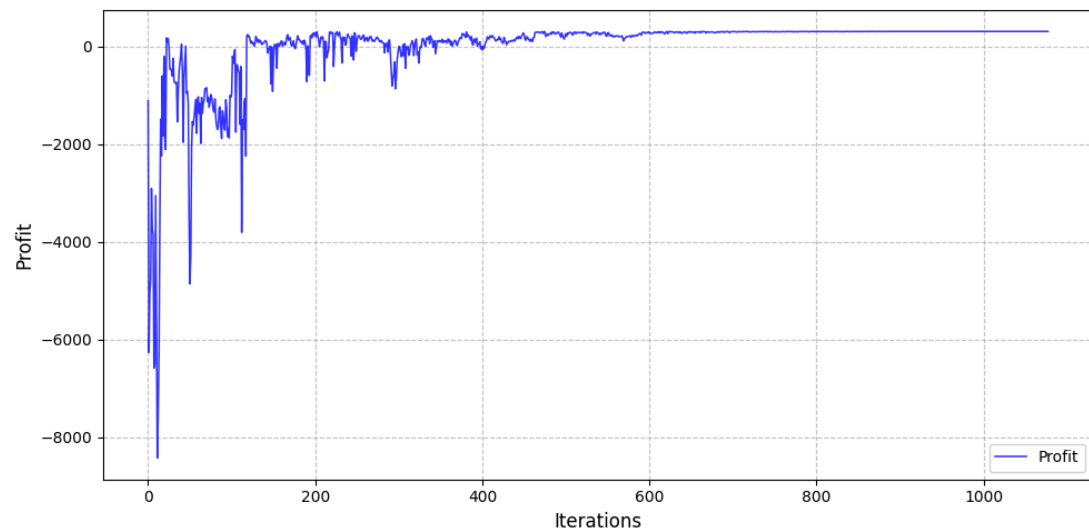


右图: *SA* 算法单次模拟库存变化曲线

🔥 初期温度较高，SA算法倾向于探索，利润大幅震荡。

📦 随着迭代次数增加，温度下降，利润逐渐收敛

📊 下图展示了不同层级的 z^m 值随温度下降从震荡到收敛的过程。

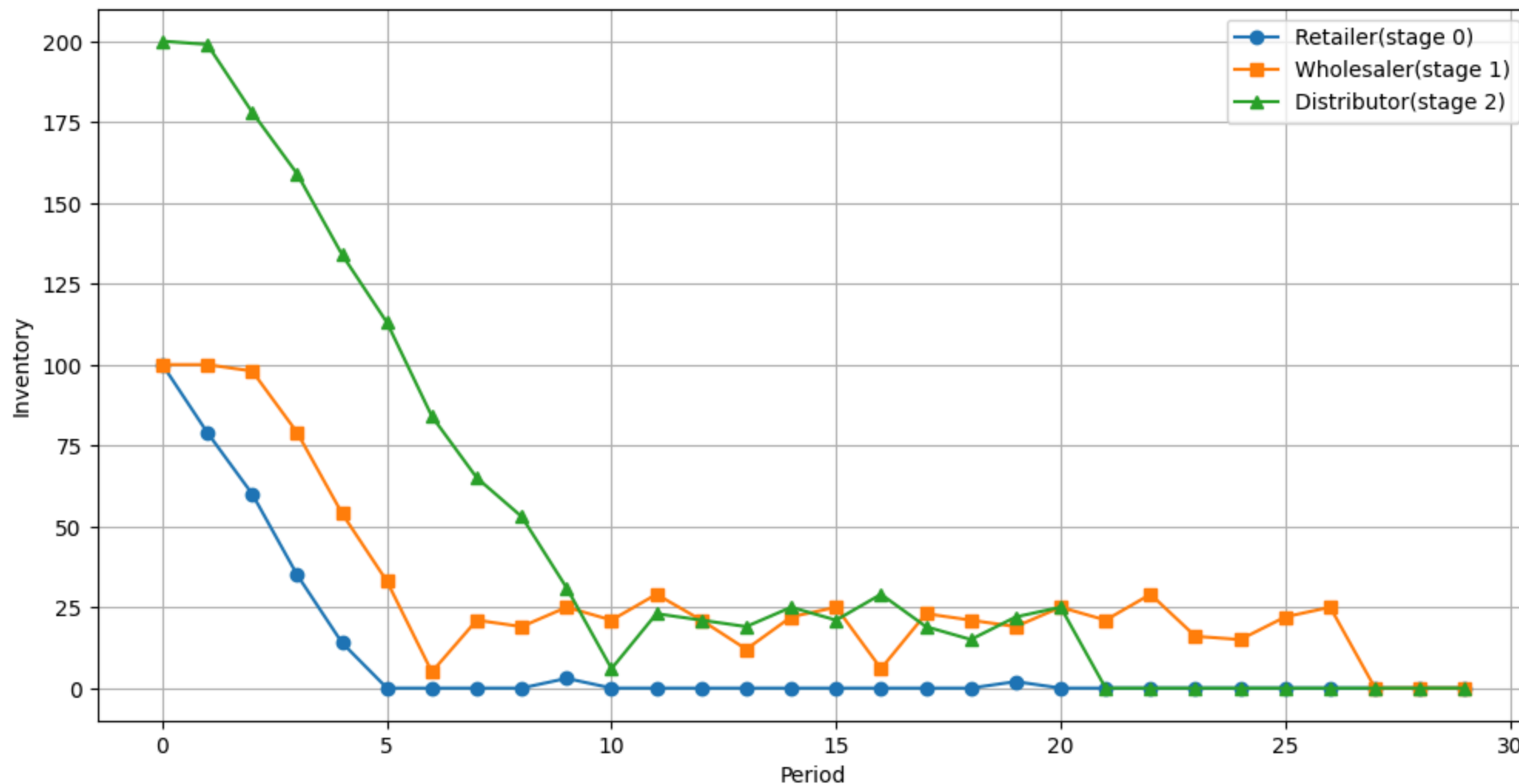


💡 在当前的每一个时间步，假设未来所有的随机需求都等于其数学期望值。

- 针对剩余的 Shrinking horizon 建立并求解一个最大化利润的全局线性规划(LP)问题。
- 求解完成后，仅提取并执行第一步的订货决策。所有未来的决策将被舍弃。
- 时间推进，环境状态更新后，重复上述求解过程。

```
import pulp  
model = pulp.LpProblem("SHLP", pulp.LpMaximize)
```

This approach requires **prior knowledge** of the demand probability distribution and assumes the expected value of the demand for all time periods



In the absence of targeted base stock levels, the requested reorder quantity $\hat{R}_t^m$ becomes the same as the accepted reorder quantity R_t^m .

- **Actor:** 根据当前环境状态提取特征并生成动作

*The actor learns a **probabilistic policy** that produces a probability distribution over available actions. This distribution is sampled from during training to encourage sufficient exploration of the state space.*

- **Critic:** 评估当前状态的价值，并根据真实奖励更新网络的参数

*The critic learns the value of each state, and the difference between the **predicted value** from the critic and the **actual rewards** received from the environment is used in the loss function ($L(\theta)$) to **update** the parameters of the networks.*

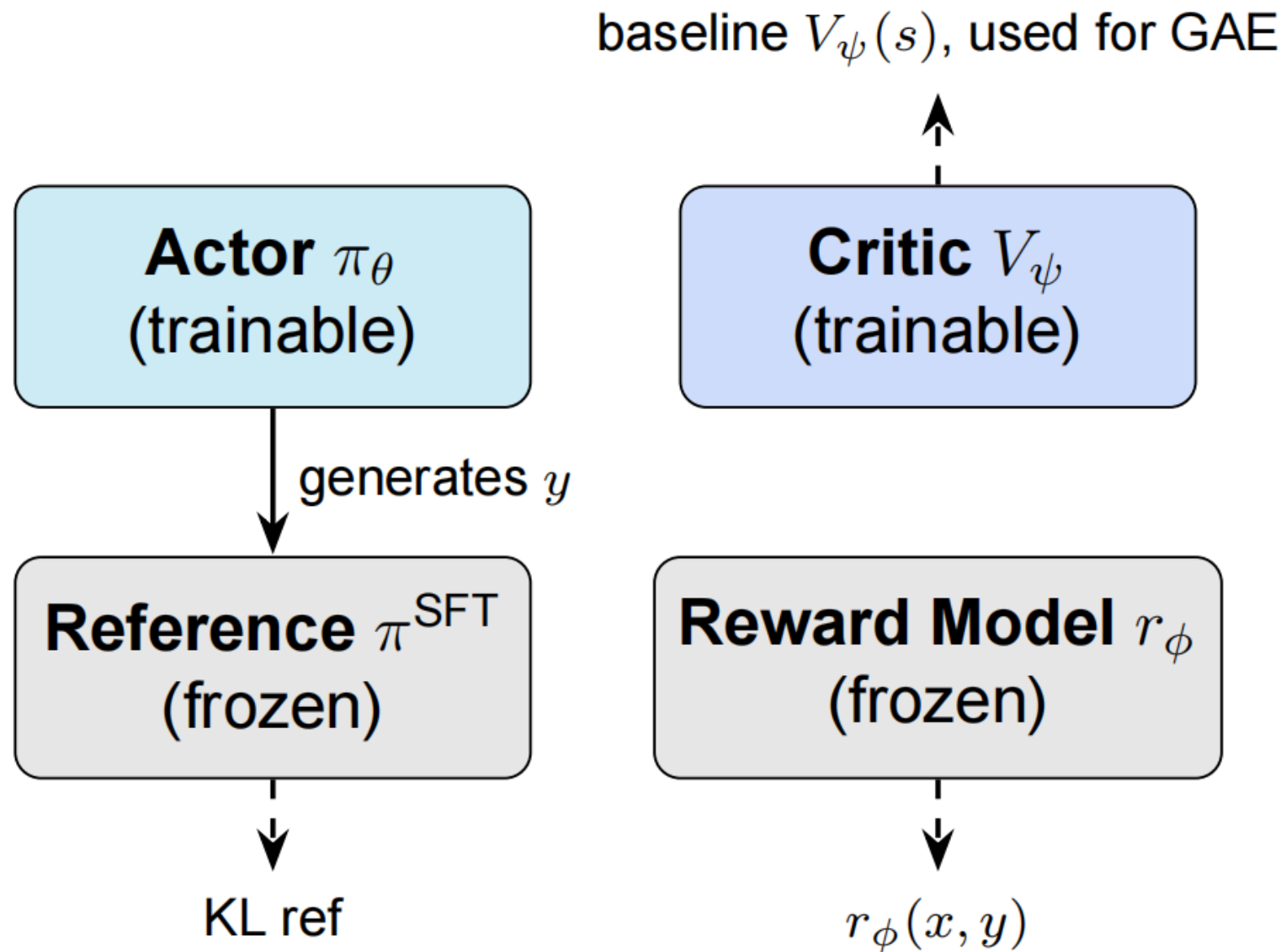
Policy Loss(Actor):

$$\mathcal{L}(\theta) = \min(r_t(\theta), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)) \hat{A}_t$$

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

$$\hat{A}_t = \sum_{t=1}^T \gamma^{T-t+1} \delta_t$$

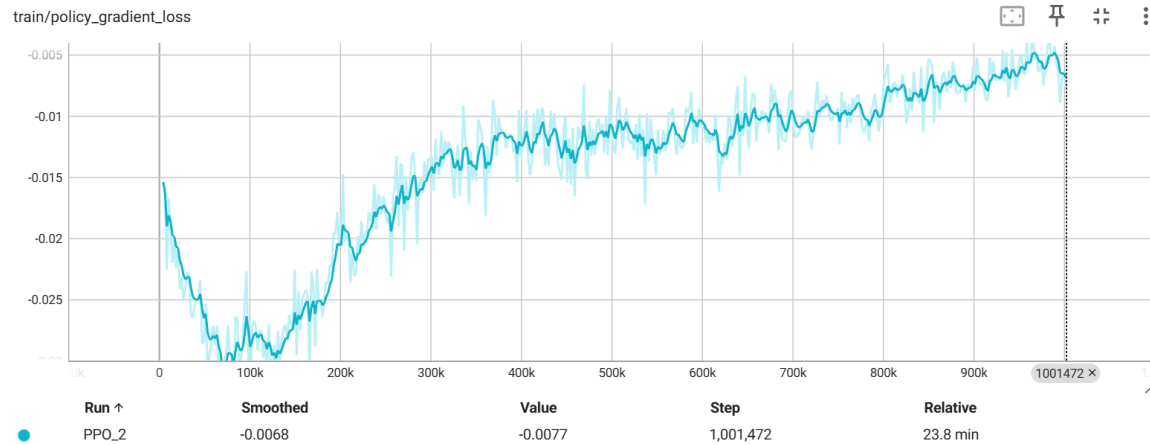
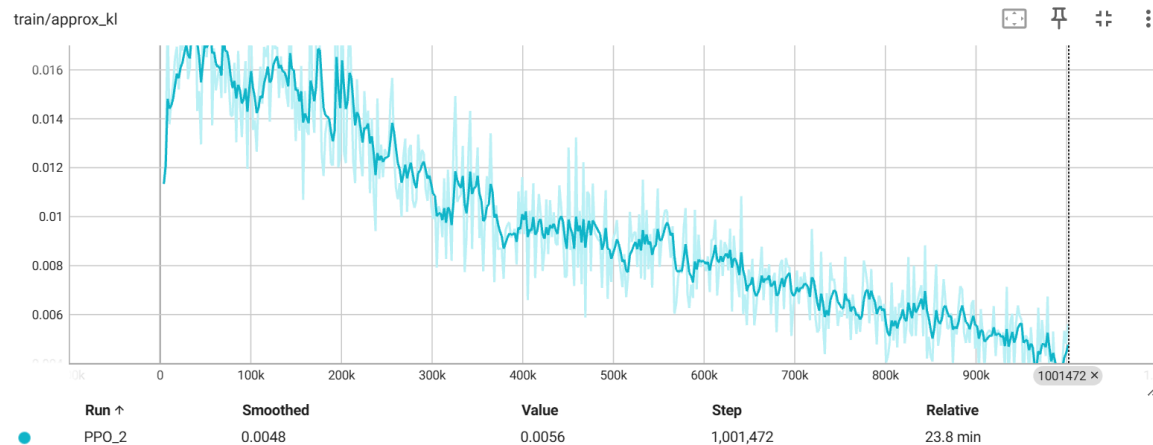
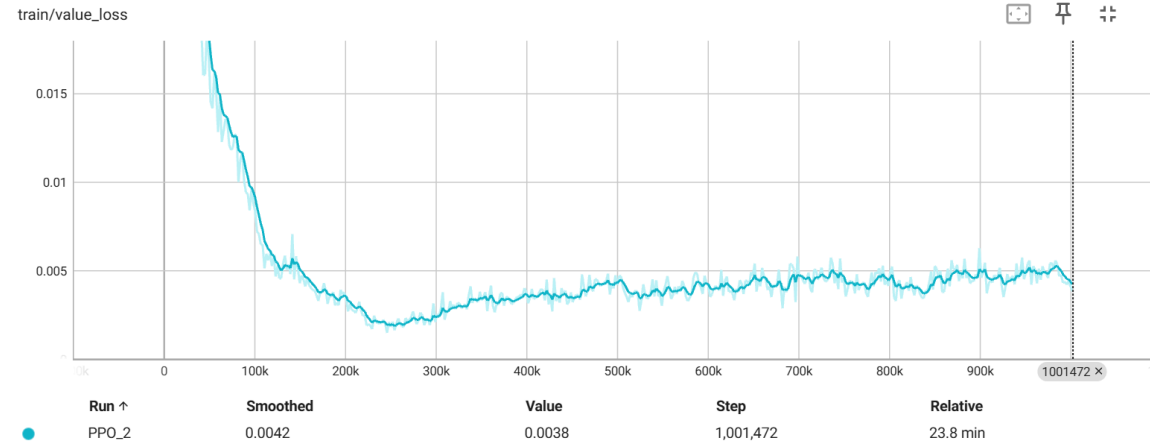
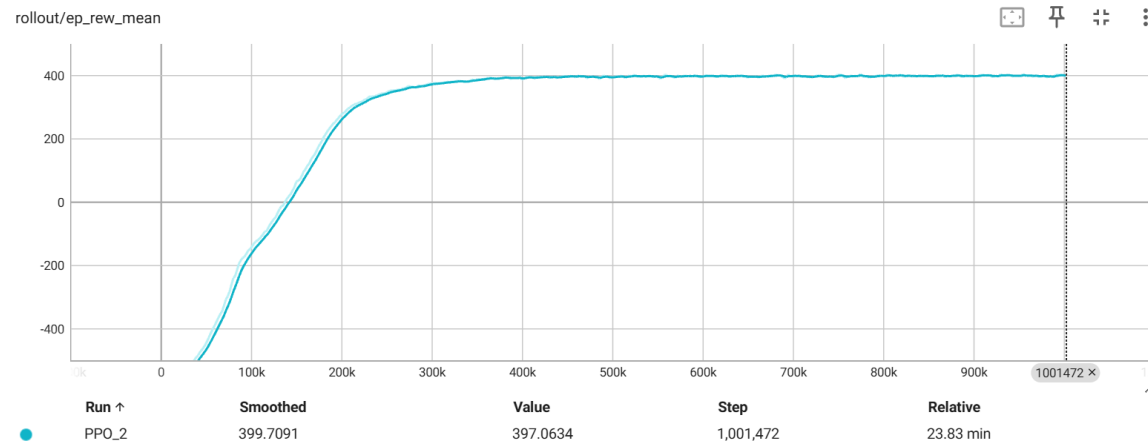
$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$



```
from stable_baselines3 import PPO
model = PPO(
    "MlpPolicy", vec_env, verbose=1,
    learning_rate=lambda progress: 3e-4 * (0.1 + 0.9 * progress), # 学习率衰减
    n_steps=2048, # on-policy 算法更新网络需要收集数据
    batch_size=512, # mini-batch gradient descent
    n_epochs=10, gamma=0.97,
    gae_lambda=0.95, # GAE bias-variance trade-off
    clip_range=0.2, # 新旧策略差异
    ent_coef=0.05, # 熵系数, 越大越鼓励探索
    vf_coef=0.5, # c1
    policy_kwargs=dict(net_arch=[256, 256]), tensorboard_log="./ppo_tensorboard/")
```

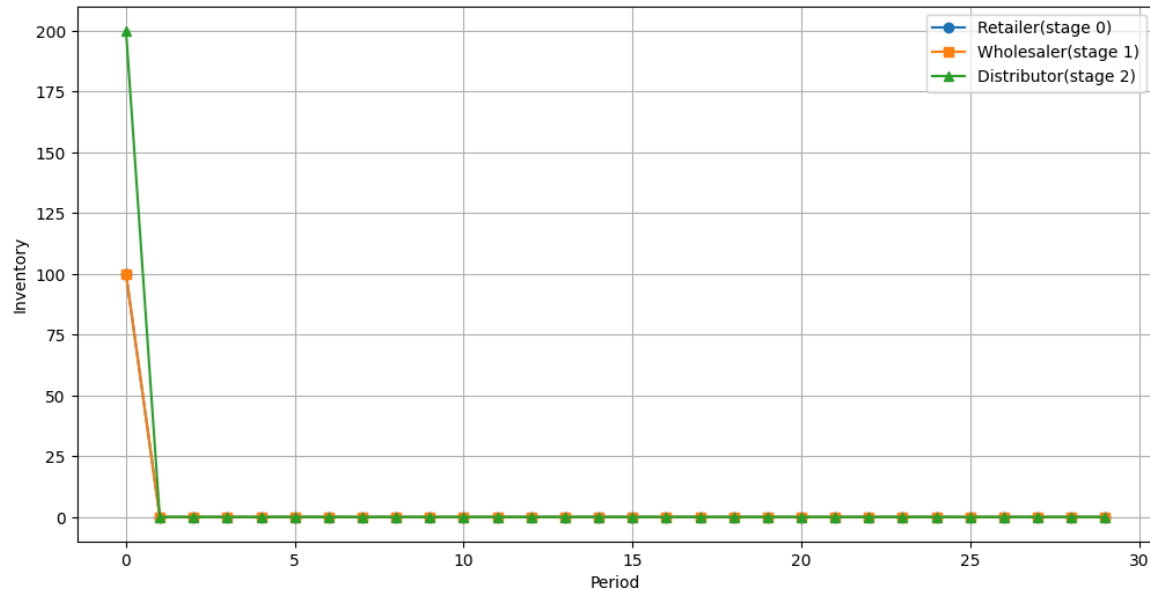
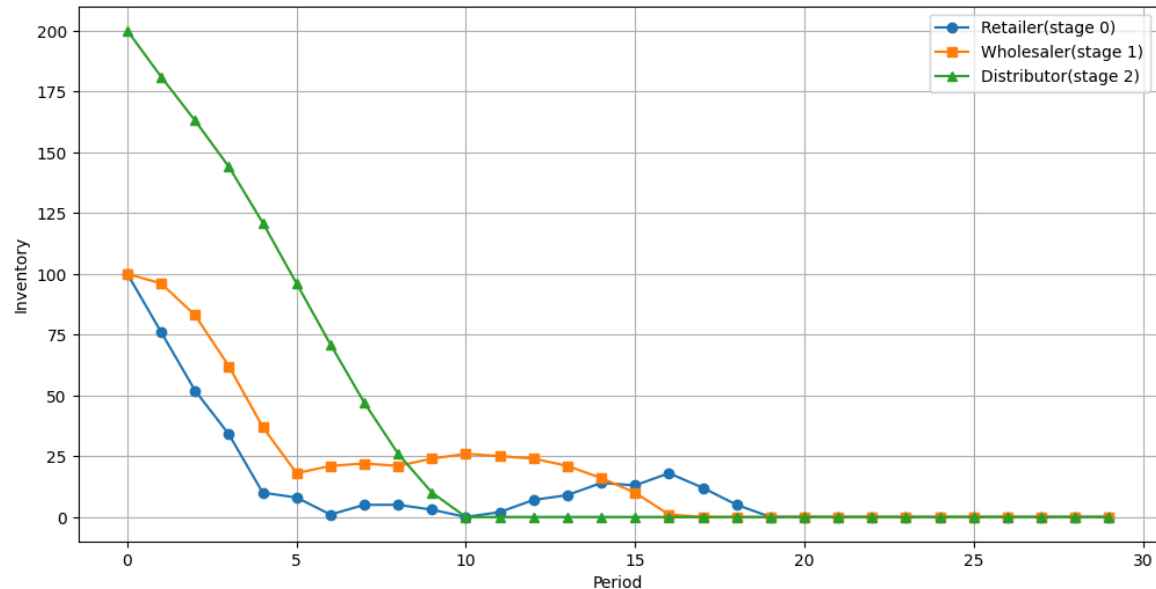
- 引入 VecNormalize 对状态和奖励进行归一化, 加速神经网络收敛

🎯 损失函数: $L_{total} = L_{policy} + c_1 L_{critic} - ent_coef \times S$



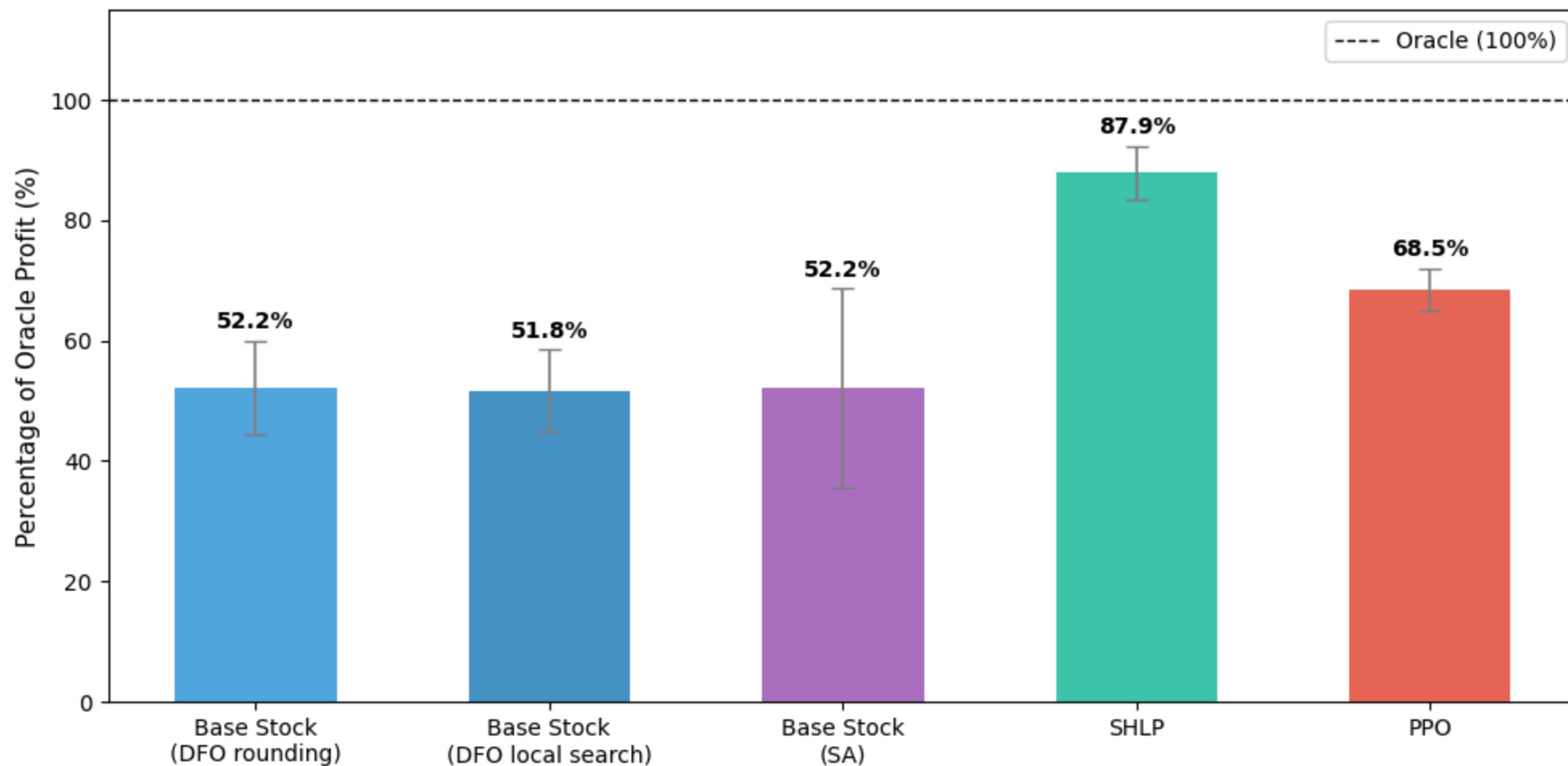
左上: ep rew mean 左下: KL 右上: value loss 右下: policy gradient loss

在Oracle模型中，每一周期的真实需求都已经被提前知晓，问题从带有随机性的动态规划问题，退化为一个个确定性的多阶段全局最优化(LP)问题。使用 Python 的 pulp 库来构建并求解该 LP 问题。

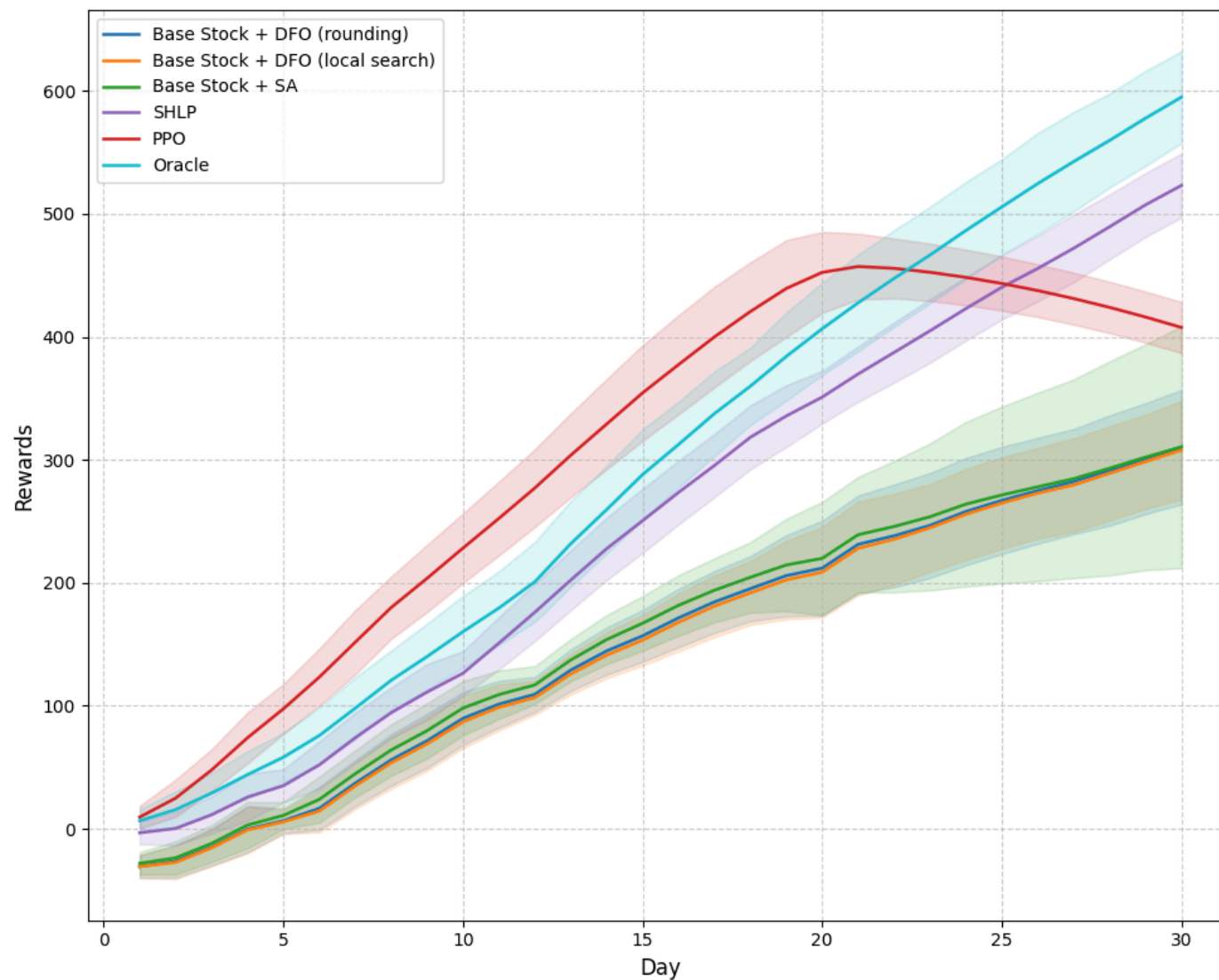


用公共随机数生成20条相同的泊松分布需求序列进行对比

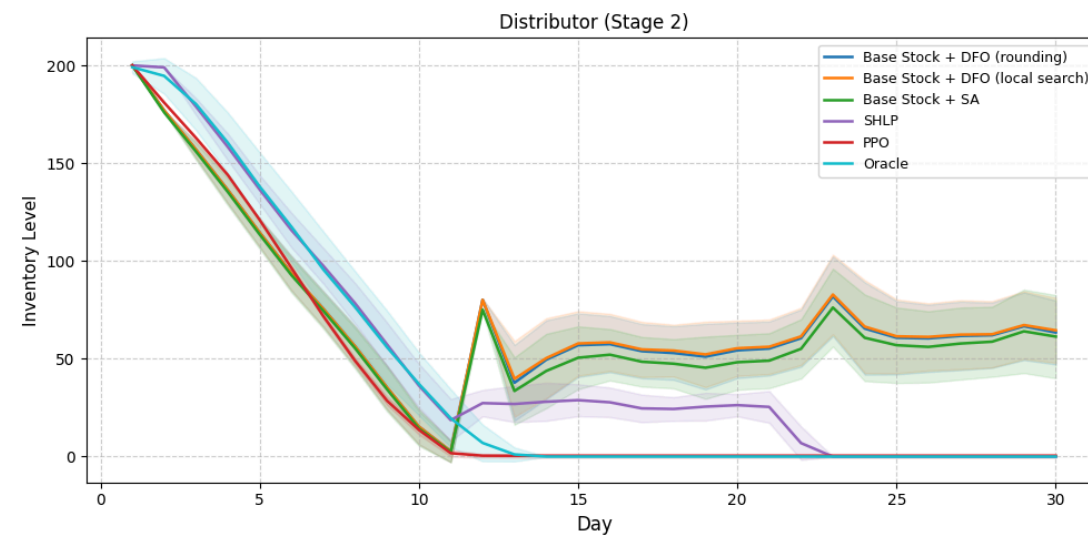
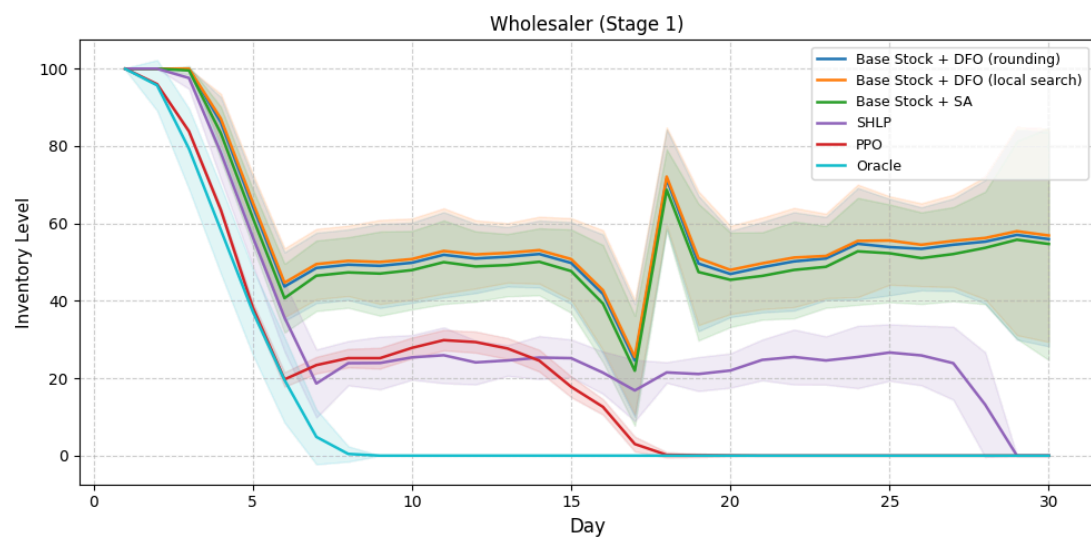
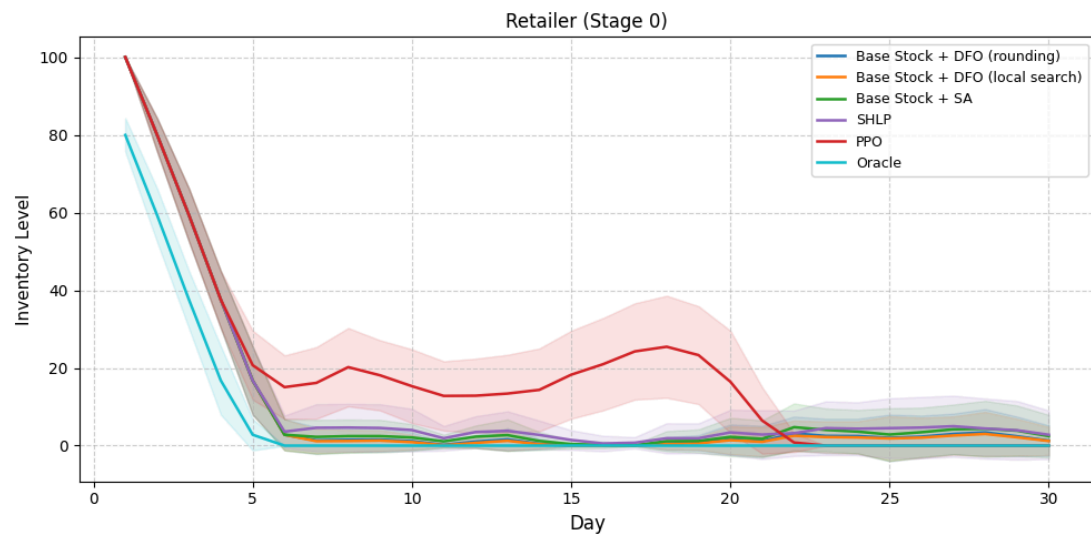
算法	平均利润	标准差	占 Oracle %
Oracle	594.86	37.30	100.00%
SHLP	523.07	25.96	87.9%
PPO	407.62	20.68	68.5%
Base Stock + DFO (rounding)	310.40	46.52	52.2%
Base Stock + DFO (local search)	307.84	40.00	51.8%
Base Stock + SA	310.38	98.34	52.2%



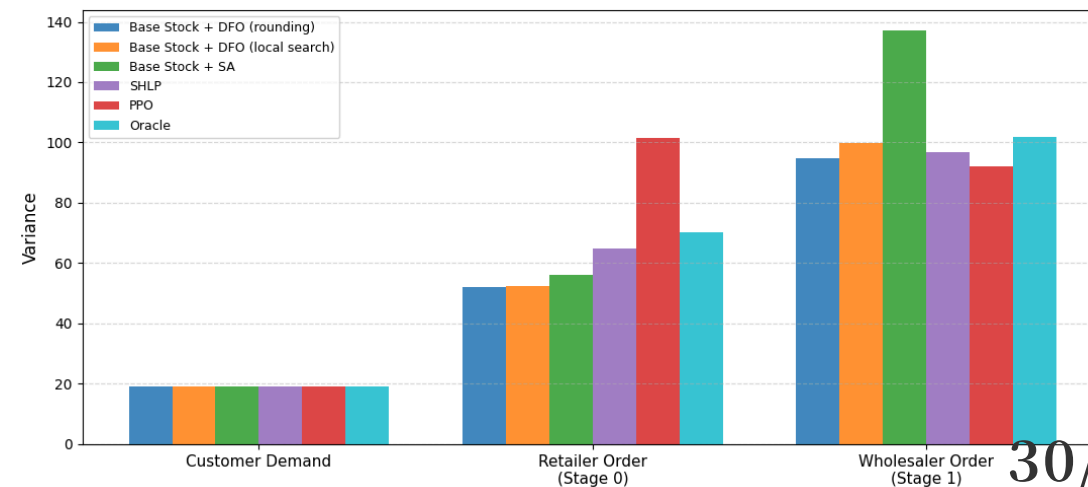
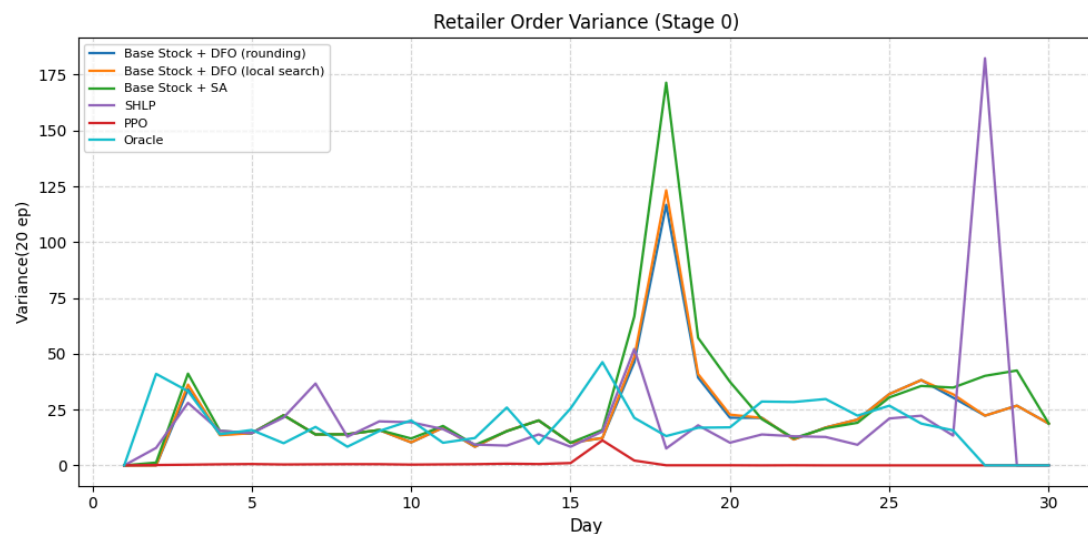
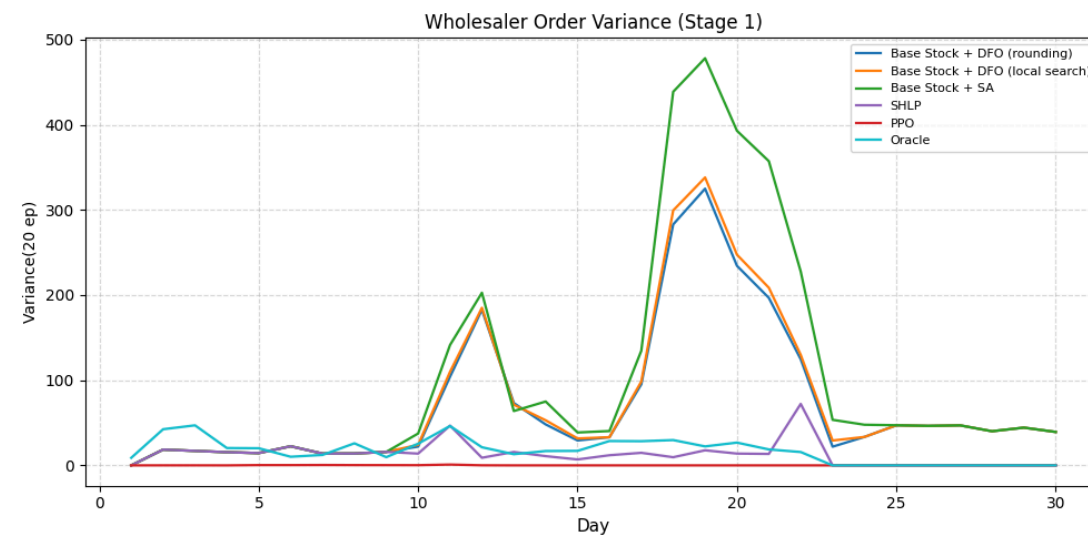
各算法平均利润与Oracle模型的比较



图：不同模型累积利润(20个ep的平均值)随时间变化的曲线。阴影表示标准差。



模拟数据：牛鞭效应



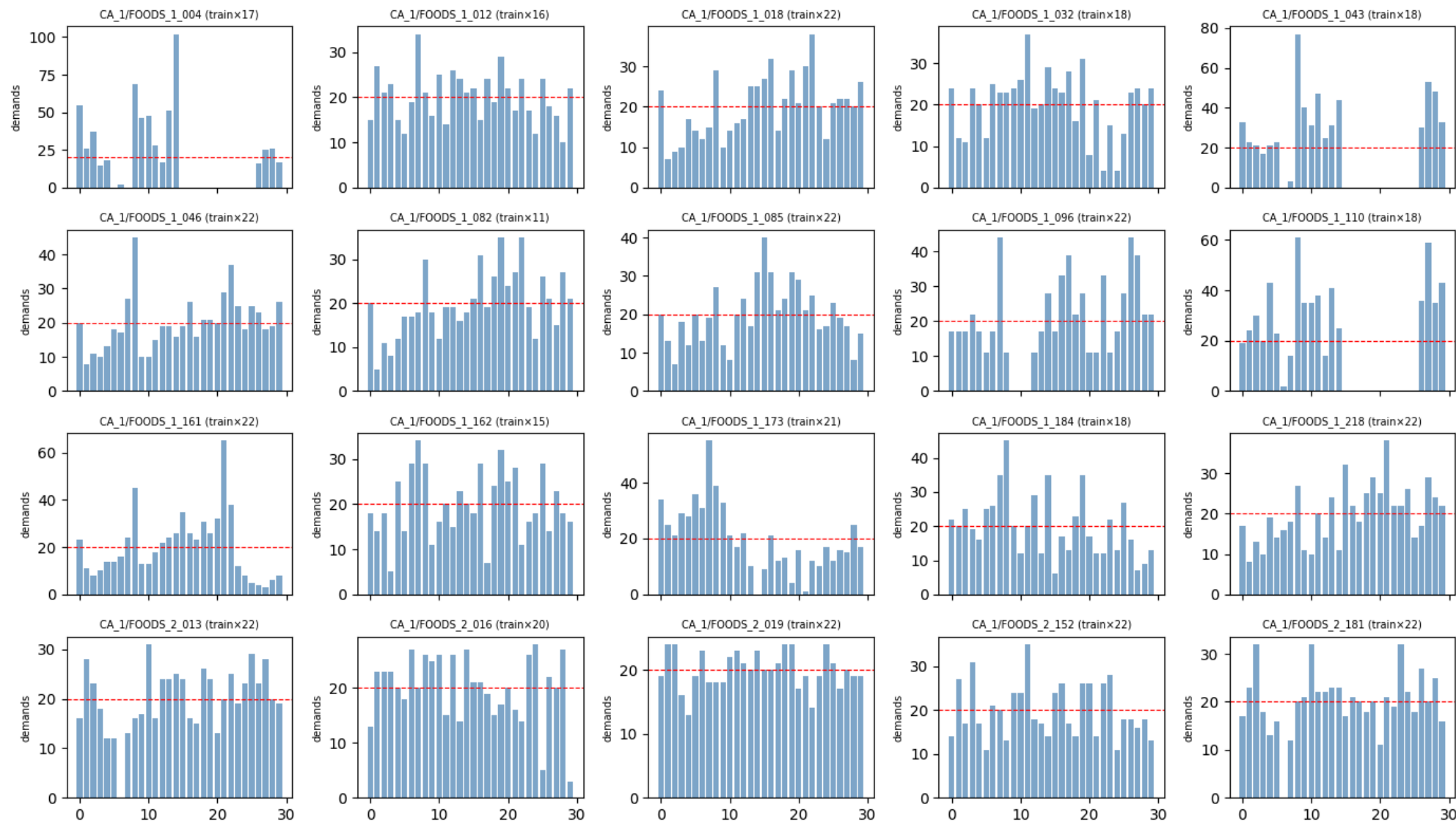
算法	T = 30	T = 50	T = 100
Oracle	0.861	1.147	1.632
SHLP	23.876	40.372	107.273
PPO	0.264	0.547	0.707
Base Stock + DFO (rounding)	0.023	0.045	0.075
Base Stock + DFO (local search)	0.026	0.046	0.077
Base Stock + SA	0.028	0.045	0.076

🤔 时间复杂度可能是 SHLP 算法的瓶颈

不同算法在不同周期上的运行时间对比，单位为秒(s)。

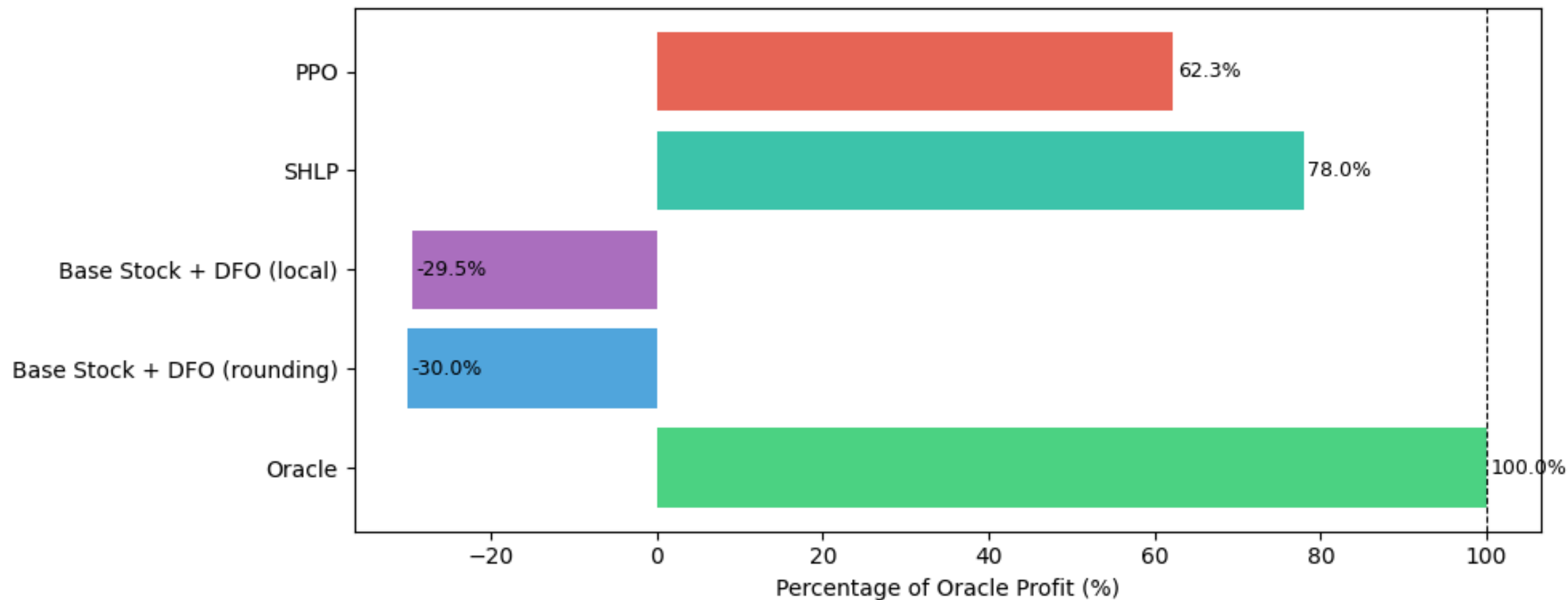
从 **Kaggle M5 Forecasting** 数据集中提取真实的沃尔玛商品周销量数据，在真实需求下对比各算法表现。

- 将周销量线性缩放到**均值 ≈ 20** ，使其与环境的成本参数匹配（价格、仓储成本、缺货惩罚、产能），保留真实数据的波动特征
- 使用滑动窗口(stride = 10)，将每个商品的时间序列划分为约20个以30周为周期的训练集，最后30周划分为测试集，避免数据泄露
- Base stock 策略需要在**训练集**上寻找最优参数 z^m ，其余算法直接在**测试集**上测试



算法	平均利润	标准差	占Oracle%
Oracle	582.68	66.47	100.0%
SHLP	454.21	57.36	78.0%
PPO	362.81	51.28	62.3%
Base Stock + DFO (rounding)	-174.90	644.68	-30.0%
Base Stock + DFO (local)	-172.04	634.76	-29.5%

注：由于SA算法充分搜索的迭代时间较久，这里不作演示

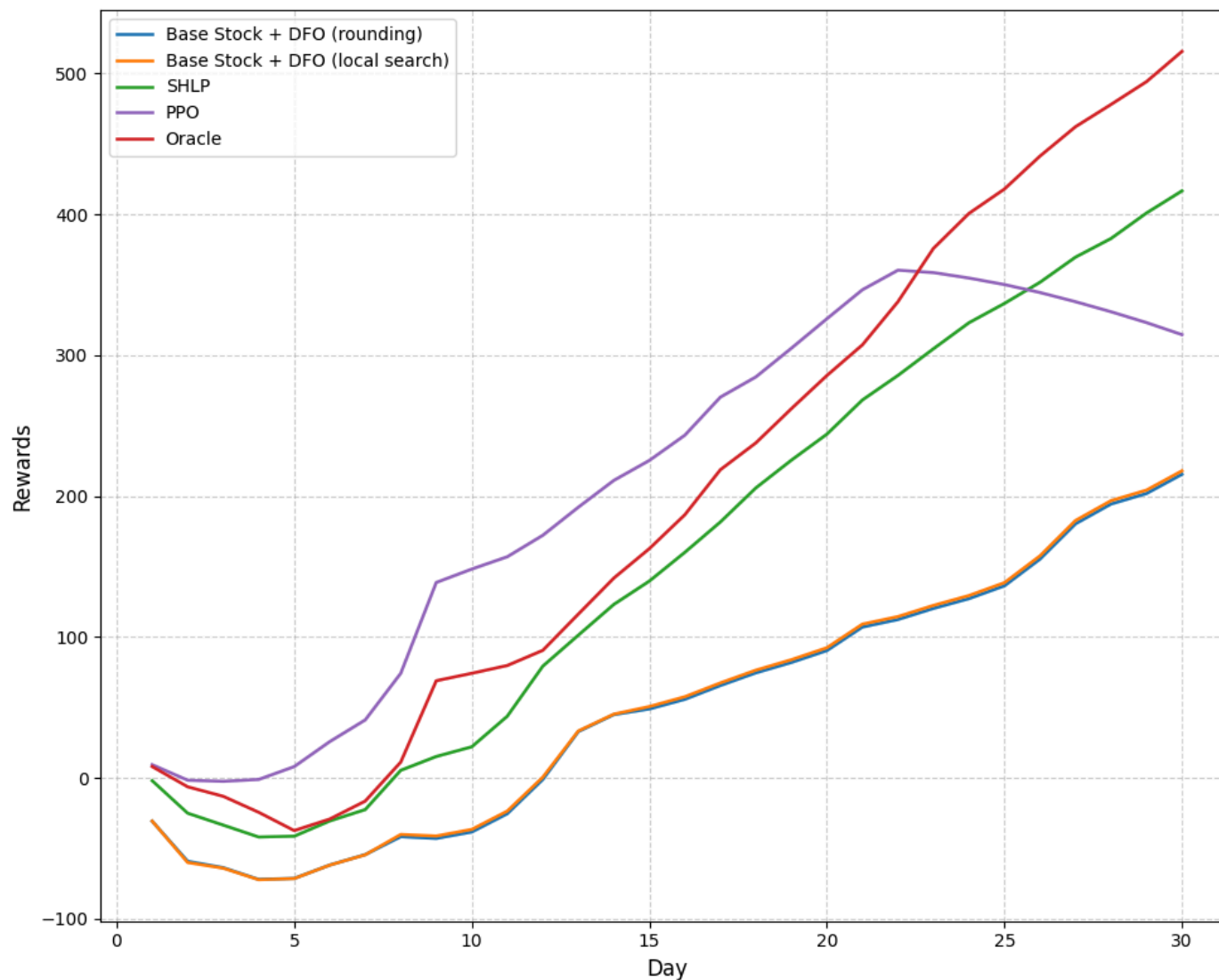


各算法平均利润与Oracle模型的比较

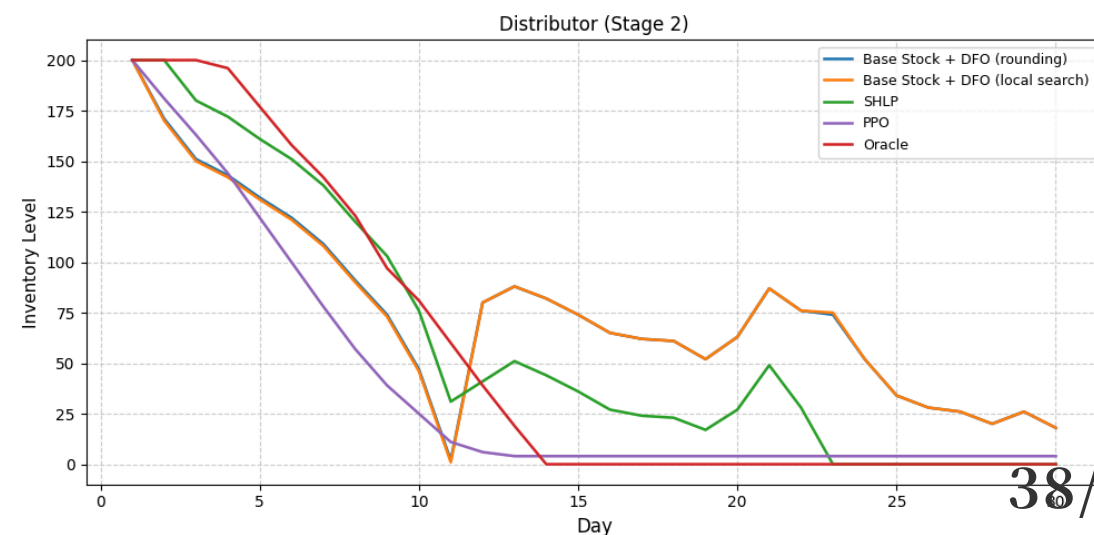
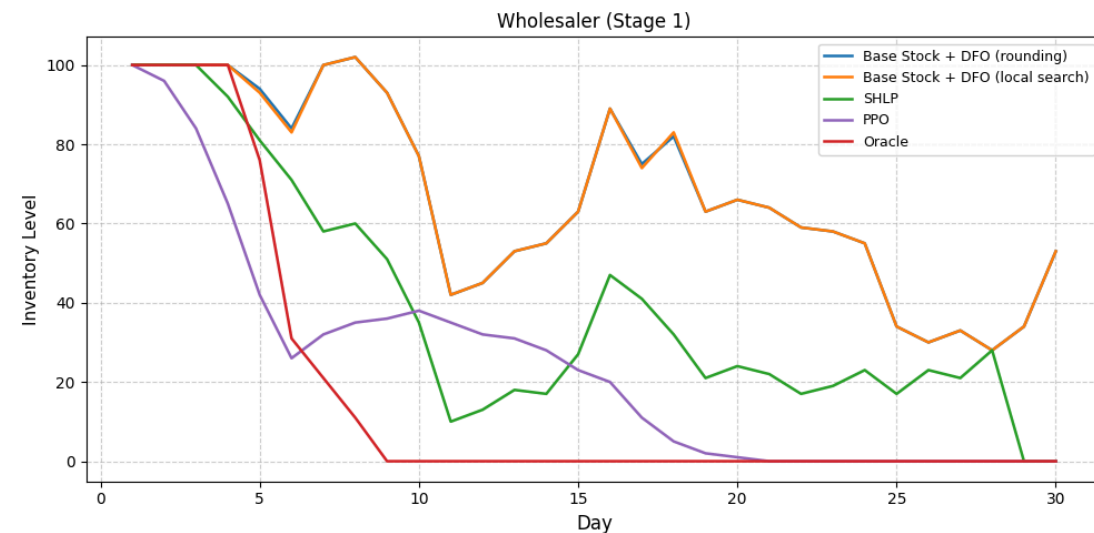
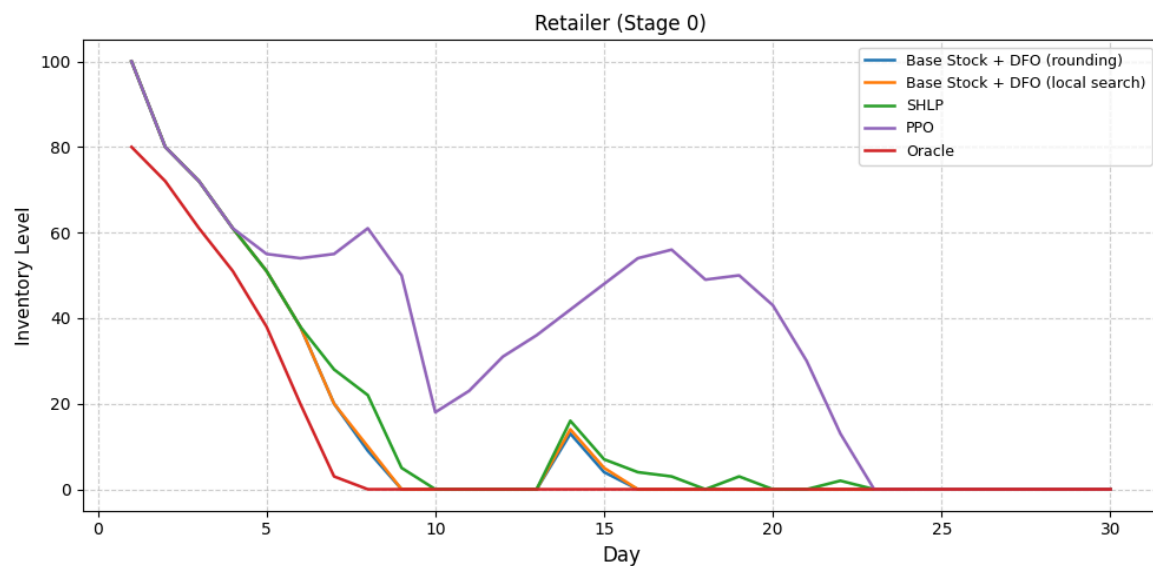
🤔 尽管Base Stock策略的平均利润是负数，并不代表对所有的商品Base Stock策略都失效 ➡ 挑选表现良好/糟糕的商品作为代表进行观察分析

商品	Oracle	DFO-R	DFO-L
CA_1/FOODS_1_012	604.0	244.4	247.6
CA_1/FOODS_1_018	495.0	123.7	126.5
CA_1/FOODS_1_032	602.3	-133.5	-135.4
CA_1/FOODS_1_043	657.5	-533.2	-530.1
CA_1/FOODS_1_046	515.6	215.5	217.9

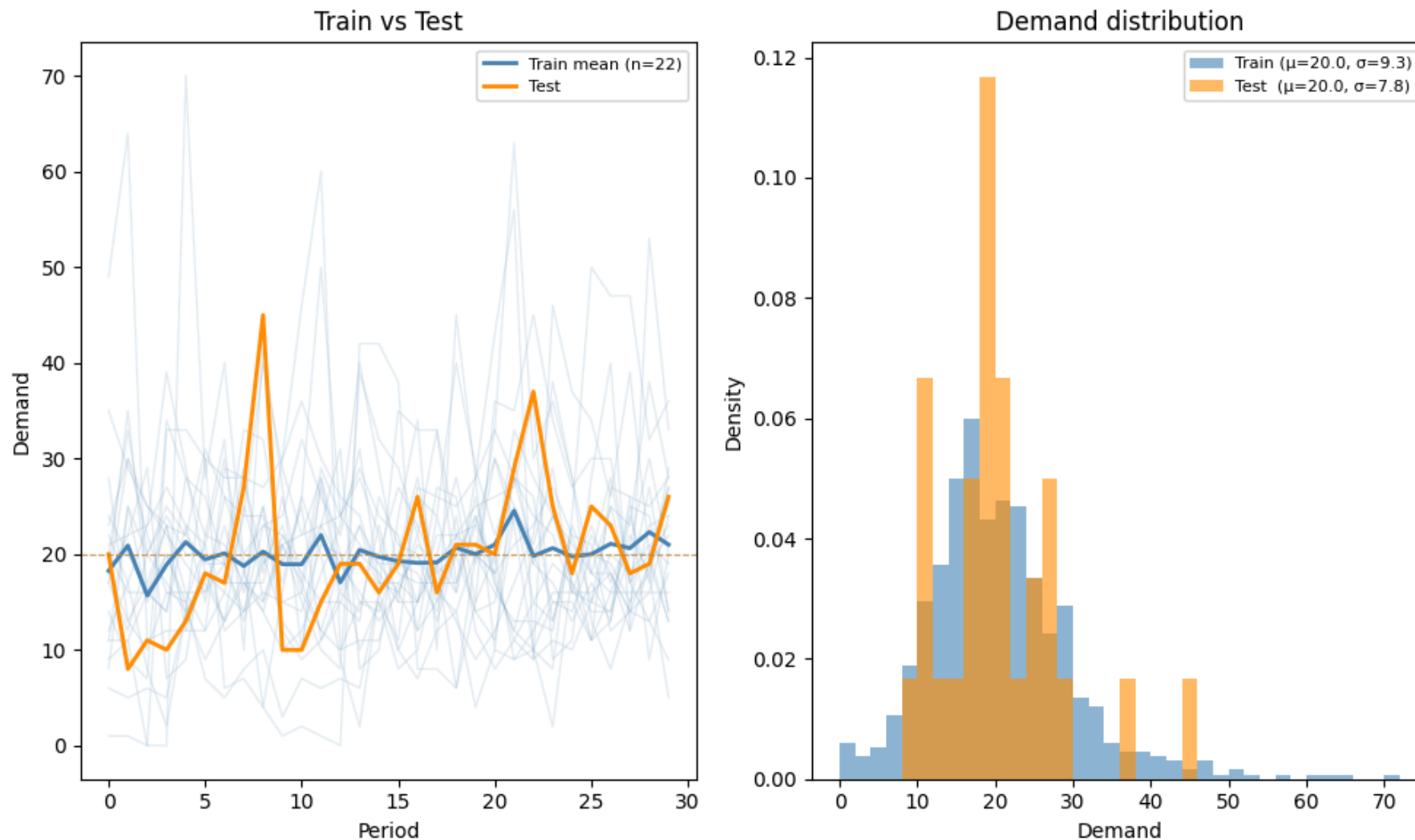
接下来选择 FOODS_1_043 和 FOODS_1_046 两个商品进行分析

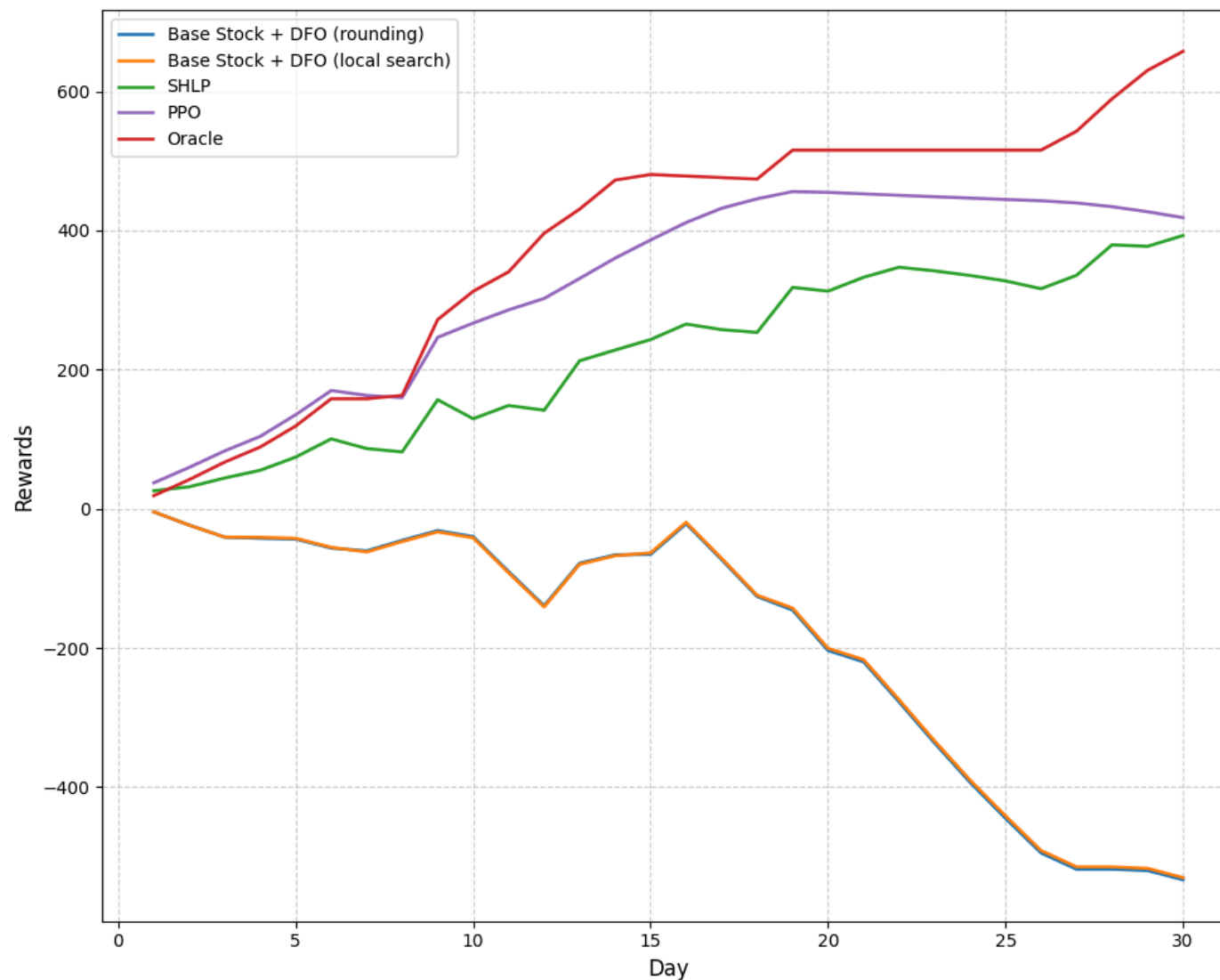


Base Stock失效原因分析：表现良好的例子

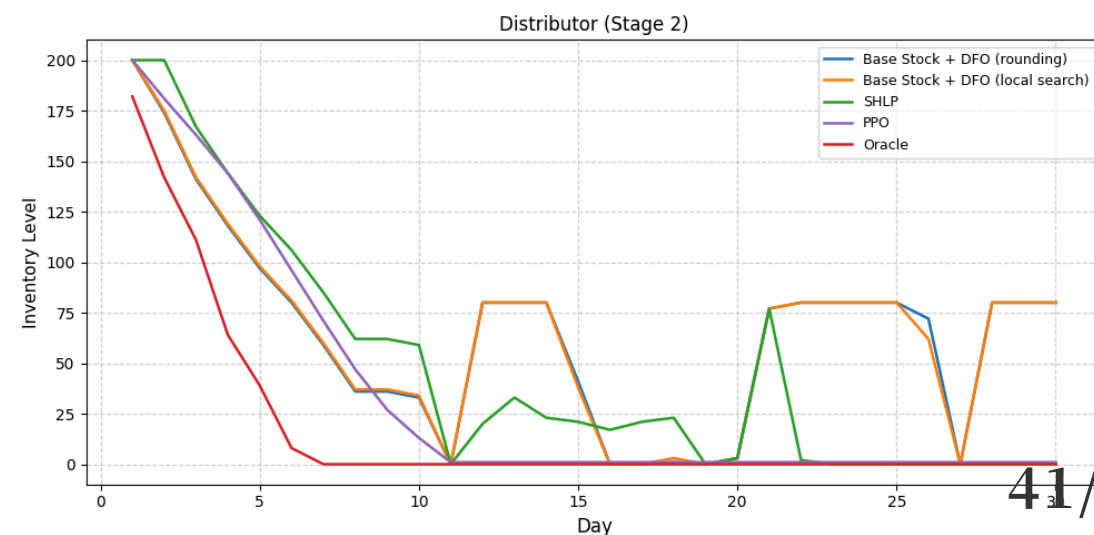
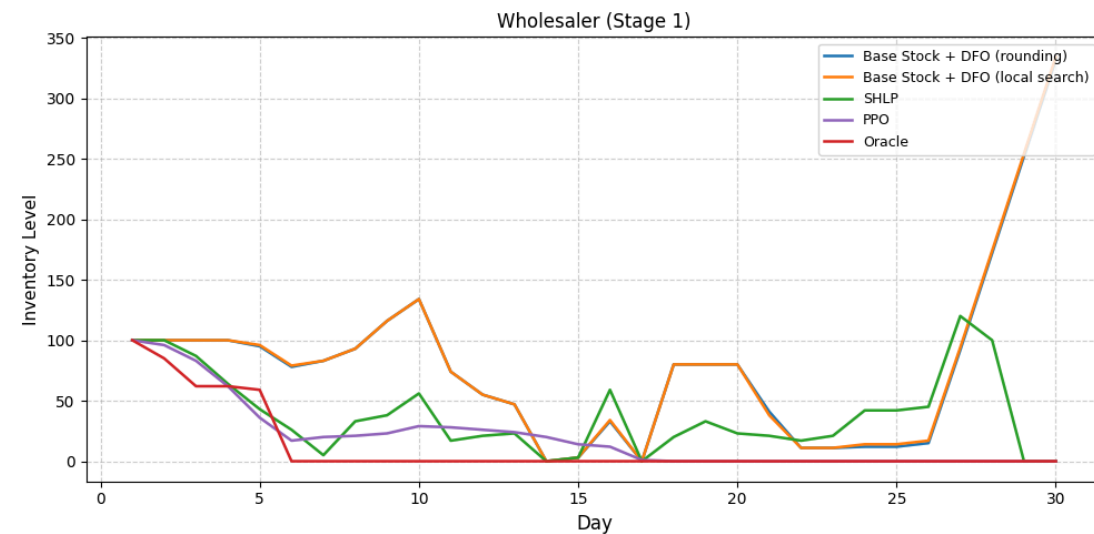
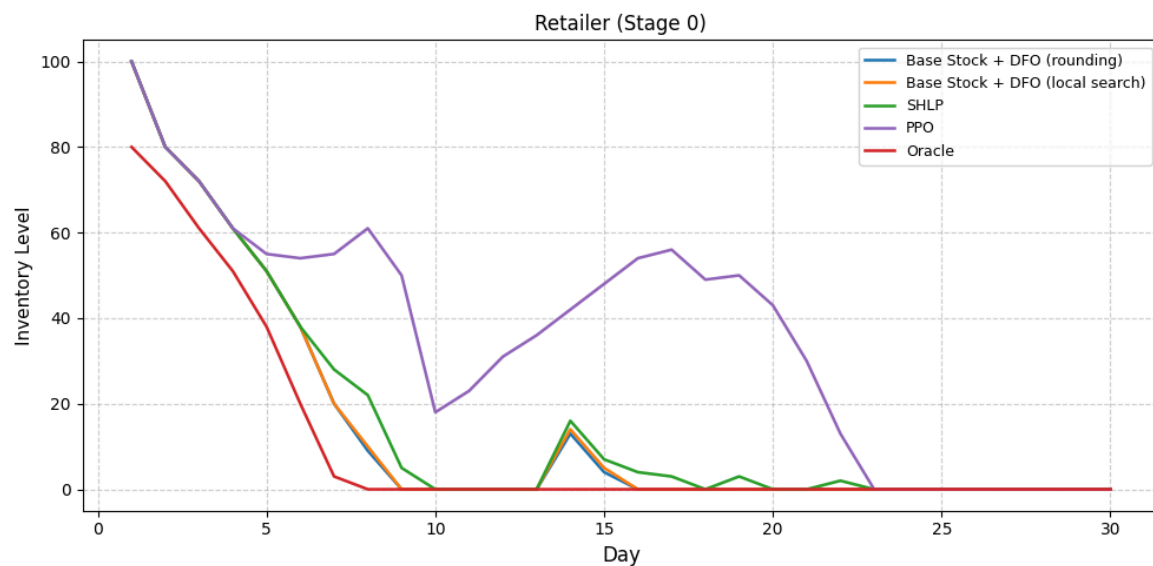


商品FOODS_1_046：各节点库存随时间变化的曲线

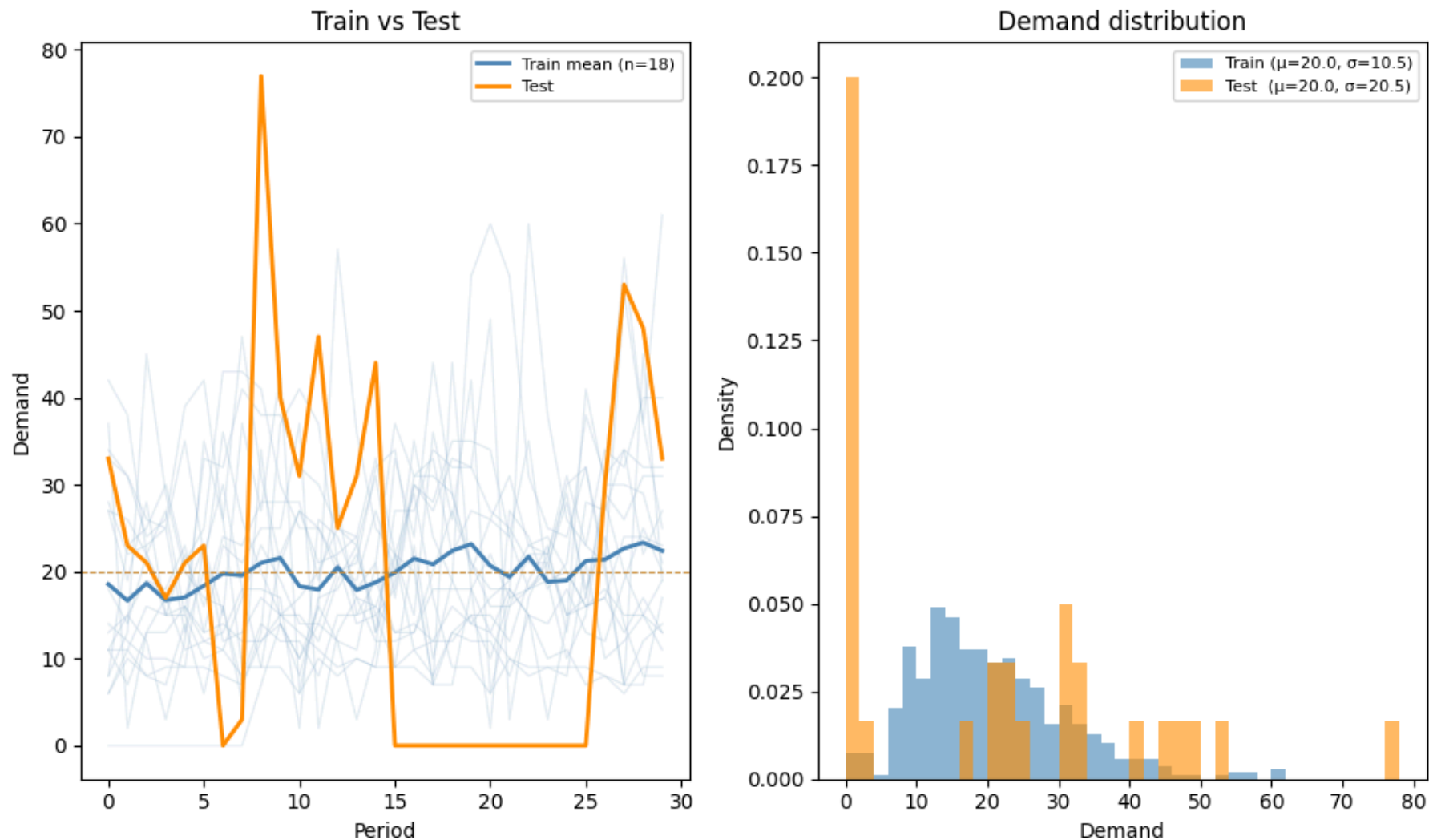




Base Stock失效原因分析：表现糟糕的例子



商品FOODS_1_043：各节点库存随时间变化的曲线



✗ 动作空间归一化：（严重下降）

PPO 默认输出的动作是从均值为 0、标准差为 1 的正态分布中采样的，而动作空间可能在 $[0, 100]$ 。在环境中统一映射到 $[-1, 1]$ 能帮助神经网络平稳收敛。

难以实现订货量为0（输出为-1） / 初始化输出在0附近导致订货过多

✗ 超参数调优：（略微下降）

增大 `n_steps`：模型更新网络前收集更多步数，有利于减少梯度方差

线性衰减 `ent_coef`：训练前期鼓励探索，后期降低熵奖励减少随机性

✓（微弱提升）

在状态空间中传入Inventory Position(IP):

$$IP_t^m = I_t^m + T_t^m - B_{t-1}^m$$

✓（略有提升：407 ➡ 410）

在最后一步为期末剩余库存添加 剩余库存奖励:

$$r_T = r_T^{\text{原始}} + \alpha^T \sum_{m=0}^{M-2} v_m \cdot I_{T+1}^m$$

v_m 设为各层级的采购成本（即库存至少值其采购价），这样 PPO 就不会在周期末端主动清空库存（事实上仍会清空）。

- ? 利用求出的 Base-Stock 策略最优解，在 RL 训练的前期，计算 RL 动作与 Base-Stock 动作的 MSE，作为辅助负奖励，强迫它先学习差不多的策略，再去微调
- ? 特征工程：在状态空间中增加相对时间特征、过去销量和库存等信息
- ? 拆分为3个独立的PPO智能体，分别负责三个层级
- ? 修改奖励函数，惩罚订货量的不平稳等
- ? 调参：调高 GAE λ ，更依赖真实的reward

📌 模型性能: $\text{SHLP} > \text{RL(PPO)} > \text{Base Stock}$

🌟 在期末清空库存前, PPO 模型的累积利润接近甚至超过 SHLP/Oracle

🤔 在现实应用中, RL 模型可以在连续时间的环境中重复训练, 以避免这一问题

The RL model could be trained on a continuous environment that cycles month after month, drawing demand from the same probability distribution to avoid this behavior in a real-world application.

💡 可以在更长的时间周期中运行模型/算法, 截取中间某个时间段比较性能